תרגיל בית 5 – מבוא למדעי המחשב מ'/ח'

סמסטר חורף 2025/6

0. הנחיות כלליות לתרגיל

- מתרגלים אחראיים: **נגה אפק בקר ועמית מלאך**
- מועד אחרון להגשה: **יום חמישי ה-29.01.2026 בשעה 23:59**
- נושאי התרגיל: **מיון, רקורסיה ו-backtracking – מצגות הרצאה 10-13 ומצגות תרגול 10-13**
- ספריות מותרות לשימוש: **`stdio.h, stdbool.h, stdlib.h` בלבד**
- ההגשה היא בבודדים בלבד!
- קראו את השאלות בעיון לפני שתתחילו בפתרונן.
- הקפידו לתעד את הקוד שלכם בהערות באנגלית.
- מלבד מילואים, לא יתקבלו תרגילים אחרי מועד הגשה. הגשה לאחר מועד הגשה נחשבת כאי־הגשה.
- מספר ימי ההגשה באיחור המותרים הם מספר ימי המילואים. על מנת לקבל את הדחייה, עליכם לעדכן במייל את המתרגל האחראי על התרגיל על יציאתכם למילואים (בטרם יציאתכם למילואים או עד שבוע לאחר פרסום התרגיל), ולאחר חזרתכם עליכם לשלוח אישור שמראה שהייתם במילואים (טופס 3010). מספר ימי המילואים הם מספר ימי הדחייה שיינתנו להגשת התרגיל.
- בדיקת התרגיל היא אוטומטית – הקפידו להדפיס בדיוק בפורמט שהתבקשתם. בפרט, ודאו שהאותיות הגדולות והקטנות המודפסות בפועל תואמות לדוגמאות הפלט הנתונות, וכן ודאו שלא מודפסים רווחים או ירידות שורה שלא התבקשתם להדפיס.
- את תרגיל הבית יש לפתור בסביבת CLion, אותה יש להתקין לפי ההנחיות שמופיעות באתר הקורס. בפרט, יש להגדיר בפרויקט את הדגלים המתאימים למהדר. לכל שאלה, קובץ ההגשה יהיה קובץ יחיד בעל סיומת C בלבד, ולא בתוך קובץ ארכיון (מסוג zip).
- ההגשה הינה אלקטרונית דרך Gradescope. לאחר ההגשה, יש להמתין ולוודא שקובץ ההגשה עובר בהצלחה את הבדיקות שפורסמו.
- חל איסור מוחלט להעתיק פתרונות מסטודנטים אחרים. המגישים של עבודות עם דמיון מעל 80% יועלו לוועדת משמעת.
- הקפידו לכתוב את הפתרונות תוך עמידה בהנחיות לכתיבת קוד (Style) שבהן השתמשנו הסמסטר. יורדו נקודות על אי־עמידה בהנחיות אלו.

1. קפיברות ויראליות – מיונים והבנת קוד

1.1 הוראות

חוות הקפיברות "CapyGram" מנהלת חשבונות ברשתות חברתיות לכל אחת מהקפיברות שלה. כל קפיברה פעילה בפלטפורמה אחת או יותר ומקבלת ציון פופולריות על בסיס מספר העוקבים, לייקים ושיתופים.

כתבו תוכנית הקוראת מערך של מספרים שלמים בטווח [0-100] המזהה ציוני פופולריות של קפיברות, ומערך נוסף המזהה את הפלטפורמה הראשית של כל קפיברה (כאשר Instagram מזוהה ע"י 0, TikTok ע"י 1 ושתי הפלטפורמות יחד מזוהות ע"י 2). התוכנית תמיין את הקפיברות כך שיהיו מקובצות לפי פלטפורמה, וכל קבוצת פלטפורמה תהיה ממוינת לפי פופולריות יורדת. יש לממש (או לתקן) את הפונקציות עם החתימות הבאות:

```
void viralCapySort(int popularity[], int platform[], int n);
```

הפונקציה מבצעת את המיון לפי פופולריות ולפי הפלטפורמה.

```
void swapCapybaras(int popularity[], int platform[], int i, int j);
```

הפונקציה מבצעת החלפה של הפופולריות והפלטפורמה.

```
bool shouldSwapCapybaras(int pop1, int plat1, int pop2, int plat2);
```

הפונקציה מבצעת בדיקה האם יש לבצע החלפה ומחזירה true במידה ויש צורך לבצע החלפה ואחרת false.

1.2 אופן הטיפול בקלט ופלט

מסופק לכם קובץ שלד, אנא השלימו את הפתרון שלכם שם. **שימו לב חלק מן הפונקציות כבר ממומשות עבורכם אך בקוד הנתון קיימות שגיאות – עליכן לתקן אותן.**

פונקציית main בקובץ קוראת מהמשתמש את המספר n שמהווה את מספר הקפיברות בחווה, ואז n מספרים שלמים שמהווים את תוכן מערך הפופולריות ולבסוף קולטת n מספרים שמהווים את הנוכחות של הקפיברות בפלטפורמות השונות.

בסוף, main תדפיס שתי שורות, הראשונה שמכילה את תוכן המערך הפופולריות הממויין והשני הממוין לפי תדירות שהתקבל, כאשר כל שני איברים מופרדים ברווח ושני המערכים מופרדים בשורה.

ניתן להניח שהקלט תקין.

1.3 דוגמאות

קלט	פלט
5 85 45 92 30 88 1 0 1 0 0	88 45 30 92 85 0 0 0 1 1
6 67 71 55 78 60 50 2 2 0 2 1 0	55 50 60 78 71 67 0 0 1 2 2 2

5 40 50 60 70 80 1 1 1 1 1	80 70 60 50 40 1 1 1 1 1

1.4 דרישות סיבוכיות

סיבוכיות זמן: $O(n^2)$

סיבוכיות מקום נוסף: $O(1)$

1.5 הערות

- המיון צריך להיות מהפופולריות הגבוהה ביותר לנמוכה ביותר בתוך פלטפורמה מסויימת (סדר יורד) ועל הפלטפורמות להיות ממויונות מהמזהה הנמוך ביותר לגבוה ביותר (סדר עולה).
- חובה להשתמש בכל פונקציות העזר ולתקן אותן במידת הצורך.
- השגיאות בקוד יכולות להופיע בכל אחת מהפונקציות בשלד **כולל** ()main.

2. Backtracking

2.1 הוראות

בתרגיל זה נכתוב קוד שיודע לפתור לוחות סודוקו!
לוח סודוקו הוא טבלה בעלת 9 שורות ו-9 עמודות. הוא מחולק ל9 תיבות קטנות של 3 על 3, כמתואר באיור.
בחלק מהתאים של הטבלה כתובות ספרות מ 1 עד 9. שאר התאים ריקים (תא ריק מיוצג על ידי הספרה 0). המטרה היא למלא את כל התאים הריקים בטבלה עם ספרות מ 1 עד 9 תוך קיום החוקים הבאים:

- בכל שורה כל אחת מן הספרות 1 עד 9 צריכה להופיע בדיוק פעם אחת.
- בכל עמודה כל אחת מן הספרות 1 עד 9 צריכה להופיע בדיוק פעם אחת.
- בכל אחת מהתיבות הקטנות בגודל 3 על 3, כל אחת מן הספרות 1 עד 9 צריכה להופיע בדיוק פעם אחת.

להלן לוח סודוקו, כאשר הספרה 0 מציינת תאים ריקים:

```
5 3 0 | 0 7 0 | 0 0 0
6 0 0 | 1 9 5 | 0 0 0
0 9 8 | 0 0 0 | 0 6 0
---------------------
8 0 0 | 0 6 0 | 0 0 3
4 0 0 | 8 0 3 | 0 0 1
7 0 0 | 0 2 0 | 0 0 6
---------------------
0 6 0 | 0 0 0 | 2 8 0
0 0 0 | 4 1 9 | 0 0 5
0 0 0 | 0 8 0 | 0 7 9
```

מסופק לכם קובץ שלד, עליכם להשלים את הפתרון שם.
בקובץ זה main קוראת מהמשתמש לוח סודוקו ומכניסה אותו למערך דו ממדי בגודל 9 על 9 המייצג את הלוח. לאחר מכן היא מדפיסה את הלוח הלא פתור, ואז וקוראת לפונקציה שלכם עם המערך הדו ממדי שנקלט מהמשתמש.

הפונקציה שלכם מקבלת כקלט את המערך דו ממדי המייצג את טבלת הסודוקו שהmain קלט מהמשתמש. כאמור חלק מהערכים בטבלה הזו חסרים.
הפונקציה צריכה להחזיר true, אם קיים פתרון ללוח הסודוקו הנתון. **ובנוסף למלא במערך שהיא קיבלה את הפתרון במקומות החסרים.**
אחרת, הפונקציה צריכה להחזיר false.

להלן חתימת הפונקציה שעליכם לממש:

```
bool solveSudoku(int board[N][N]) {
```

ה main ידאג אח"כ להדפיס בהתאם לערך החזרה מהפונקציה שלכם:
(אתם לא צריכים לממש את הmain. הוא נתון לכם בקובץ השלד)

- אם הוחזר true יודפס:
  - Solved Sudoku:
    ואת הלוח הפתור שעדכנתם בתוך המערך
- אם הוחזר false יודפס:
  - No solution exists.

שימו לב! אתם לא צריכים להתעסק בקלט מהמשתמש וההדפסה בכלל. השלד של main עושה את כל זה בשבילכם.
אתם רק צריכים לממש את הלוגיקה המתבקשת בפונקציה solveSudoku.
מומלץ מאוד לקרוא בעיון את קובץ השלד לפני תחילת העבודה כדי להבין כיצד הוא מבצע את הקלט והפלט, ואיך הפונקציה שלכם נקראת ממנו.

2.2 הנחות

ניתן להניח שהקלט תקין – כלומר הפונקציה שלכם תמיד תקבל מערך דו ממדי, מסוג int, מגודל 9 על 9, כשבכל תא בו יש את הספרה 0 עד 9.

ניתן להניח כי כל לוח נתון הוא חוקי – כלומר לא תקבלו לוח שמאותחל בצורה שלא מקיימת את אחד מחוקי הסודוקו שהוסברו לעיל.

ניתן להניח שלכל לוח נתון יש או אפס פתרונות או פתרון אחד יחיד.

2.3 דוגמאות

להלן דוגמאות ריצה עבור לוח הסודוקו שנתון למעלה, ועבור לוח סודוקו אחר – לא פתיר:

1. עבור לוח הסודוקו שמתואר בתחילת השאלה יודפס:

```
Original Sudoku:
5 3 0 | 0 7 0 | 0 0 0
6 0 0 | 1 9 5 | 0 0 0
0 9 8 | 0 0 0 | 0 6 0
---------------------
8 0 0 | 0 6 0 | 0 0 3
4 0 0 | 8 0 3 | 0 0 1
7 0 0 | 0 2 0 | 0 0 6
---------------------
0 6 0 | 0 0 0 | 2 8 0
0 0 0 | 4 1 9 | 0 0 5
0 0 0 | 0 8 0 | 0 7 9

Solved Sudoku:
5 3 4 | 6 7 8 | 9 1 2
6 7 2 | 1 9 5 | 3 4 8
1 9 8 | 3 4 2 | 5 6 7
---------------------
8 5 9 | 7 6 1 | 4 2 3
4 2 6 | 8 5 3 | 7 9 1
7 1 3 | 9 2 4 | 8 5 6
---------------------
9 6 1 | 5 3 7 | 2 8 4
2 8 7 | 4 1 9 | 6 3 5
3 4 5 | 2 8 6 | 1 7 9
```

2. ועבור לוח סודוקו נוסף (לא פתיר) יודפס:

```
Original Sudoku:
5 3 0 | 0 7 0 | 0 0 0
6 0 0 | 1 9 5 | 0 0 0
0 9 8 | 0 0 0 | 0 6 0
---------------------
8 0 0 | 0 6 0 | 5 0 3
4 0 0 | 8 0 3 | 0 0 1
7 0 0 | 0 2 0 | 0 0 6
---------------------
0 6 0 | 0 0 0 | 2 8 0
0 0 0 | 4 1 9 | 0 0 5
0 0 0 | 0 8 0 | 0 7 9

No solution exists.
```

2.4 הערות

- יש להשתמש בשיטת backtracking כפי שנלמדה בכיתה.
- בשאלה זו אין דרישות סיבוכיות, אולם כמקובל ב-`backtracking` יש לוודא שלא מתבצעות קריאות רקורסיביות מיותרות עם פתרונות שאינם חוקיים.
- ניתן ומומלץ להשתמש בפונקציות עזר (ויש לממש את כולן).
- חובה לשחרר את כל הזיכרון שהקצתם בתום השימוש בו (במידה והקצתם זיכרון).

בהצלחה!